

VisionInspect AI

Milestone 1 — Detailed Team Progress Presentation

Presented by Om Shakthi Vemuganti • July 10, 2026

VisionInspect AI is an AI-powered manufacturing quality inspection platform. A Quality Engineer or Factory Supervisor uploads a product image; the system runs it through an anomaly-detection model trained on what a defect-free product looks like, scores how severe any detected defect is, and returns a pass/fail decision — with a dashboard for reviewing results and quality trends over time.

Four of us each owned one layer of the system end-to-end this milestone:

- Yash Goyal — the AI / anomaly-detection model
- Arnab Ghosal — the database
- Lakshyatha Chamarthy — the backend / REST API
- Himabindhu Ravuri — the frontend dashboard
- Omshakthi Vemuganti- supported teammates, and pulled everyone's work into this presentation.

2. System Architecture

Request flow: user uploads an image on the frontend → backend authenticates the request (JWT) and stores the image → image is preprocessed → the AI model runs anomaly detection → a defect is classified and severity-scored → a pass/fail decision is generated → the result is persisted in the database → the report is returned to the frontend and shown on the dashboard.

Layer	Owner / Tech
Frontend	Himabindhu — React (Vite) + Tailwind CSS + React Router + Context API
Backend / API	Lakshyatha — Flask, JWT auth, 12 REST endpoints
Database	Arnab — PostgreSQL, 5 relational tables, SQLAlchemy ORM
AI / Model	Yash — PatchCore (Anomalib), MVTec AD dataset

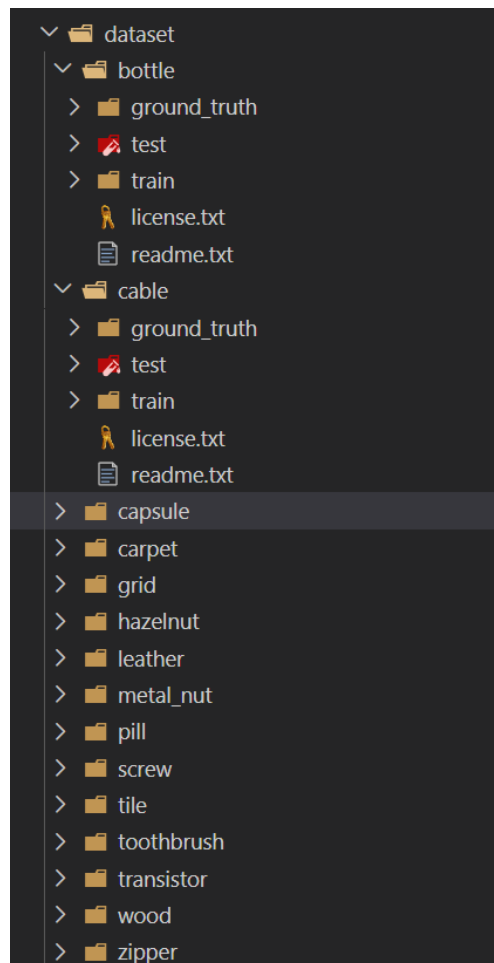
3. AI / ML Pipeline — Yash Goyal

3.1 Environment

- Python 3.10, VS Code, Windows
- Key libraries: PyTorch, Anomalib, Torchvision, OpenCV, NumPy, Pandas, Matplotlib, Pillow

3.2 Dataset

The project uses MVTec AD — an industrial anomaly-detection benchmark dataset with 15 product/texture categories, each containing normal (defect-free) training images and multiple defect-type test images with ground-truth masks.



Dataset folder structure — 15 MVTec AD categories loaded into the project

Metric	Count
Categories	15
Training images	3,629
Testing images (467 good + 1,258 defective)	1,725
Ground-truth masks	1,258

Metric	Count
Total images	5,354

3.3 Dataset Exploration

A custom script (explore_dataset.py) verified folder structure, counted images per category, identified defect types, checked resolution/color channels, and flagged corrupted files — confirming the dataset was clean before any preprocessing began.

```
Category : bottle
-----
Training Images      : 209
Good Test Images     : 20
Defective Test Images : 63
Total Test Images    : 83
Ground Truth Masks   : 63
Image Resolution     : 900 x 900
Color Channels       : 3 Channels
Corrupted Images     : 0

Defect Types:
broken_large: 20
broken_small: 22
contamination: 21
```

Per-category exploration output (example: Bottle category)

```
Overall Dataset Summary
Total Categories      : 15
Training Images      : 3629
Testing Images       : 1725
Good Test Images     : 467
Defective Test Images : 1258
Ground Truth Masks   : 1258
Corrupted Images     : 0
Total Images         : 5354
```

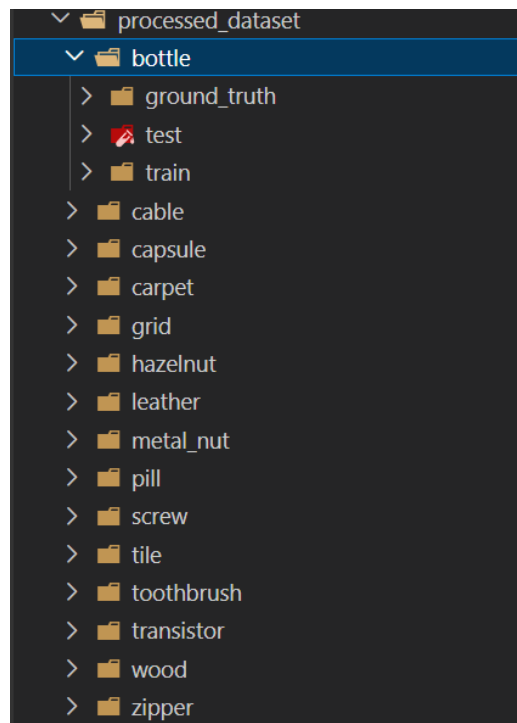
Overall dataset summary across all 15 categories

3.4 Preprocessing

- Validated dataset structure, detected and skipped corrupted/unreadable files
- Resized all images to 256×256; masks resized using INTER_NEAREST (preserves hard mask edges)
- Original folder structure preserved in a new `processed_dataset/` directory
- He skipped other preprocessing as PatchCore's built-in PreProcessor already handles normalization, and the dataset is already high-quality, so these steps would have removed useful defect detail

```
=====
Processing : grid
Processing : hazelnut
Processing : leather
Processing : metal_nut
Processing : pill
Processing : screw
Processing : tile
Processing : toothbrush
Processing : transistor
Processing : wood
Processing : zipper
=====
Dataset preprocessing completed successfully.
(yony) PS C:\Projects\VisionInspect-AT\AT\src\data>
```

Preprocessing running across all 15 categories



processed_dataset/ — same structure preserved after preprocessing

3.5 Model Selection & Configuration

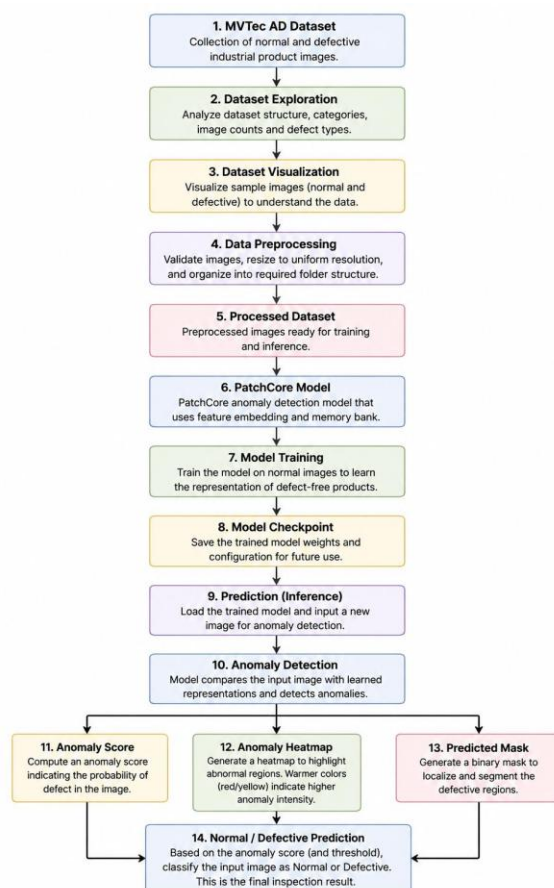
PatchCore (from the Anomalib framework) was selected because it only needs normal images to train — no defect-labelled data required — and it provides both an anomaly score and a localization heatmap.

Parameter	Value
Backbone	Wide ResNet50-2 (pretrained)
Feature Layers	Layer 2, Layer 3
Coreset Sampling Ratio	0.1

Parameter	Value
Nearest Neighbours (k)	9
Input Size	256 × 256
Batch Size (train / eval)	32 / 32

How it works: PatchCore extracts deep features from normal training images and builds a “memory bank.” At inference, it compares a new image's features against that memory bank using nearest-neighbour search — the further away a match is, the higher the anomaly score.

3.6 Full AI Pipeline



14-step AI pipeline: dataset → preprocessing → PatchCore → training → inference → anomaly score / heatmap / mask → Normal or Defective

3.7 Model Training (Proof of Concept — Bottle Category)

Although full training is scheduled for Milestone 2, Yash trained and validated the complete pipeline end-to-end on one category (Bottle) now, to prove the approach works before scaling to all 15.

	Name	Type	Params	Mode	FLOPs
0	pre_processor	PreProcessor	0	train	0
1	post_processor	PostProcessor	0	train	0
2	evaluator	Evaluator	0	train	0
3	model	PatchcoreModel	24.9 M	train	0

Trainable params: 24.9 M
 Non-trainable params: 0
 Total params: 24.9 M
 Total estimated model params size (MB): 99.450
 Modules in train mode: 19
 Modules in eval mode: 174

PatchCore model summary — 24.9M trainable parameters

OUTPUT	DEBUG CONSOLE	PROBLEMS	PORTS	QUERY RESULTS	TERMINAL
<pre> Selecting Coreset Indices.: 100% ##### 21379/21400 [19:03<00:01, 17.67it/s] Selecting Coreset Indices.: 100% ##### 21381/21400 [19:03<00:01, 17.46it/s] Selecting Coreset Indices.: 100% ##### 21383/21400 [19:04<00:00, 17.42it/s] Selecting Coreset Indices.: 100% ##### 21385/21400 [19:04<00:00, 17.43it/s] Selecting Coreset Indices.: 100% ##### 21387/21400 [19:04<00:00, 18.11it/s] Selecting Coreset Indices.: 100% ##### 21389/21400 [19:04<00:00, 18.39it/s] Selecting Coreset Indices.: 100% ##### 21391/21400 [19:04<00:00, 18.52it/s] Selecting Coreset Indices.: 100% ##### 21393/21400 [19:04<00:00, 17.96it/s] Selecting Coreset Indices.: 100% ##### 21395/21400 [19:04<00:00, 17.25it/s] Selecting Coreset Indices.: 100% ##### 21397/21400 [19:04<00:00, 17.06it/s] Selecting Coreset Indices.: 100% ##### 21399/21400 [19:04<00:00, 17.07it/s] Selecting Coreset Indices.: 100% ##### 21400/21400 [19:05<00:00, 18.69it/s] Epoch 0/0 7/7 0:00:27 + 0:00:00 0.26it/s `Trainer.fit` stopped: `max_epochs=1` reached. Epoch 0/0 7/7 0:00:27 + 0:00:00 0.26it/s </pre>					

Coreset index selection during training

VISIONINSPECT AI	
> .vscode	
> AI	
> dataset	
> outputs	
> models	
> Patchcore\MVTecAD\bottle	
> latest\weights\lightning	
> model.ckpt	
> v0	

Trained model checkpoint (model.ckpt) saved

3.8 Model Prediction

The trained checkpoint is loaded, a new image is preprocessed automatically, features are compared to the memory bank, and the model outputs an anomaly score, a Normal/Defective label, a heatmap, and a predicted defect mask.

Test 1 — Normal (“Good”) Image

```
(venv) PS C:\Projects\VisionInspect AI\AI\src> python -m models.predict
ated, use `instance(treespec, TreeSpec)` and `treespec.is_leaf()` instead.
C:\Projects\VisionInspect AI\venv\lib\site-packages\lightning\pytorch\trainer\connectors\data_connector.py:434: The 'predict_data_loader'
does not have many workers which may be a bottleneck. Consider increasing the value of the 'num_workers' argument' to 'num_workers=11' in
the 'DataLoader' to improve performance.
C:\Projects\VisionInspect AI\venv\lib\site-packages\torch\utils\data\dataloader.py:752: UserWarning: 'pin_memory' argument is set as true
but no accelerator is found, then device pinned memory won't be used.
super().__init__(loader)
Predicting 1/1 0:00:00 = 0:00:00 0.00it/s

Image      : 000.png
Category   : bottle
Status     : Normal
Anomaly Score : 0.2267

(venv) PS C:\Projects\VisionInspect AI\AI\src>
```

Prediction run — Status: Normal, Anomaly Score: 0.2267

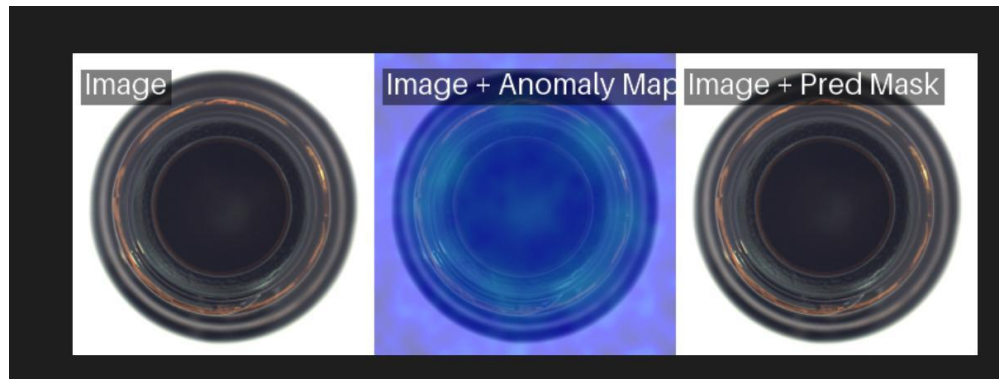


Image / anomaly map / predicted mask — correctly shows no significant anomaly

Test 2 — Defective Image

```
OUTPUT  DEBUG CONSOLE  PROBLEMS  PORTS  QUERY RESULTS  TERMINAL
(venv) PS C:\Projects\VisionInspect AI\AI\src> python -m models.predict
ated, use `instance(treespec, TreeSpec)` and `treespec.is_leaf()` instead.
C:\Projects\VisionInspect AI\venv\lib\site-packages\lightning\pytorch\trainer\connectors\data_connector.py:434: The 'predict_data_loader'
does not have many workers which may be a bottleneck. Consider increasing the value of the 'num_workers' argument' to 'num_workers=11' in
the 'DataLoader' to improve performance.
C:\Projects\VisionInspect AI\venv\lib\site-packages\torch\utils\data\dataloader.py:752: UserWarning: 'pin_memory' argument is set as true
but no accelerator is found, then device pinned memory won't be used.
super().__init__(loader)
Predicting 1/1 0:00:01 = 0:00:00 0.00it/s

Image      : 000.png
Category   : bottle
Status     : Defective
Anomaly Score : 0.942

(venv) PS C:\Projects\VisionInspect AI\AI\src>
```

Prediction run — Status: Defective, Anomaly Score: 0.942

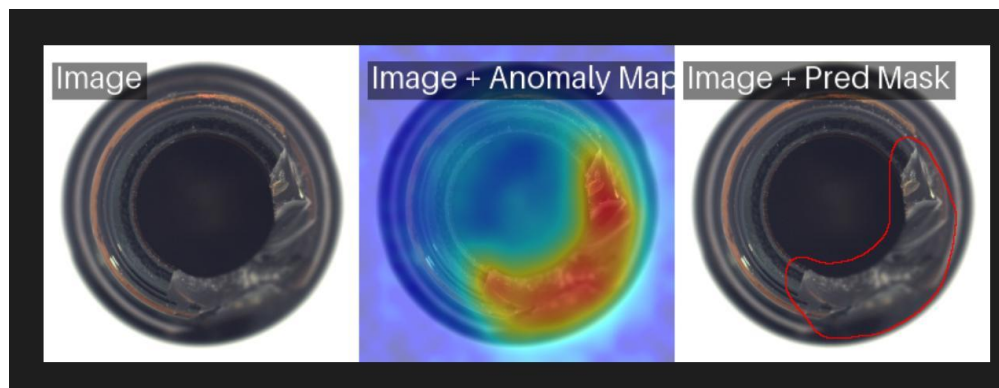


Image / anomaly map / predicted mask — heatmap and mask correctly localize the defect

3.9 Challenge Faced & Fixed

Initial predictions were wrong — good images were being flagged as defective. Root cause: normalization was applied twice (once explicitly in the code, and again automatically by PatchCore's built-in PreProcessor). Removing the duplicate normalization and retraining fixed it.

4. Database Module — Arnab Ghosal

4.1 Tools

PostgreSQL was chosen over MongoDB because the core data — users, inspections, defects, decisions — is highly relational, with clear one-to-many relationships best enforced with foreign keys.

4.2 Schema — 5 Tables

Table	Purpose
users	Login accounts & roles (quality engineer, factory supervisor, admin)
inspections	One row per uploaded product image, with status & category
defects	One row per defect found in an inspection, with severity scoring fields
quality_decisions	Final Pass / Fail / Rework decision per inspection
reports	Aggregated daily stats for the analytics dashboard

Relationships: users (1) → (many) inspections (1) → (many) defects; inspections (1) → (1) quality_decisions; reports stands alone for dashboard aggregation.

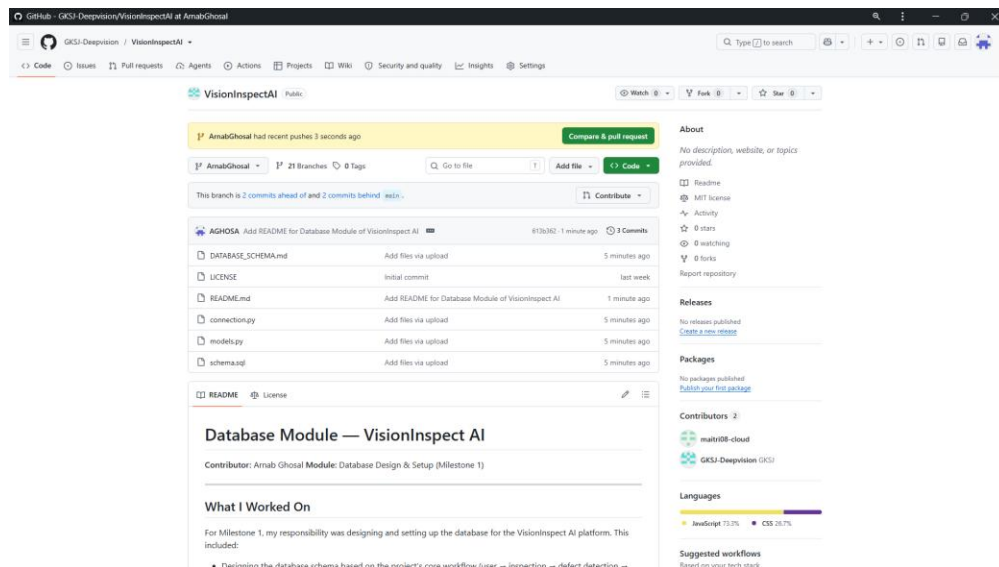
4.3 Severity Scoring — Built Into the Data Model

Not just documented — the formula is implemented as helper methods on the Defect model, so the score is calculated consistently wherever it's used.

Score Range	Level	Action
80–100	Critical	Reject product
60–79	High	Rework recommended
40–59	Medium	Review required
0–39	Low	Acceptable

4.4 Testing & GitHub

Schema was created in pgAdmin 4 against a local visioninspect_db — all 5 tables created successfully with no errors. Work was committed and pushed to the shared repository.



Database module files pushed to GitHub (GKSJ-Deepvision/VisionInspectAI)

4.5 Also Completed

- connection.py reads DB credentials from environment variables — same code works locally and after cloud deployment, no changes needed
- Handed off to backend team — integration needs only importing the db object and model classes, no schema changes required

5. Backend / API — Lakshyatha Chamarth

5.1 Environment

- Python, Flask, VS Code, Windows, Python venv
- Libraries: Flask-CORS, PyJWT, SQLite (dev), Pillow, NumPy, Pytest

Backend project structure is modular: routes, services, database, utils, and uploads are kept in separate folders, with app.py as the entry point.

5.2 REST API — 12 Endpoints Built

Endpoint	Description
POST /api/auth/register	Register a new user
POST /api/auth/login	Authenticate & issue JWT token
GET /api/auth/me	Current authenticated user profile
POST /api/upload	Upload a product image
GET /api/inspection	List inspection records
POST /api/inspection	Create a new inspection record
GET /api/inspection/<id>	Inspection detail by ID
PUT /api/inspection/<id>	Update an inspection record
GET /api/analytics	Quality analytics data
GET /api/history	Inspection history
GET /api/dataset	List dataset categories
GET /api/dataset/<category>/files	List files for a category/split

5.3 Authentication

- User registration & login implemented
- JWT tokens generated & validated via PyJWT, 24-hour expiry
- Protected endpoints (e.g. GET /api/auth/me) require a valid token

5.4 AI Integration — Scaffolded Ahead of Time

Service-layer components were already created so plugging in the real model later is a drop-in change:

- Inference service structure
- Image processing service
- Quality control service
- Severity scoring service
- Dataset support for the MVTec dataset structure

5.5 Automated Testing

Backend functionality was verified with an automated Pytest suite — not manual spot-checks.

```
Microsoft Windows [Version 10.0.26200.8737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\LAKSHYATHA\OneDrive\Desktop\VisionInspectAI\backend>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.20.10.3:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 105-864-806
```

pytest -v — 10/10 backend tests passed (auth, inspection, upload, dataset)

All 10 tests passed: user registration, login, profile retrieval, inspection create/read/update, image upload, and dataset category/file listing.

5.6 Integration Readiness

- Routes connected to service modules, services connected to the database
- Verified ready for frontend integration
- API contract documented for the frontend team.

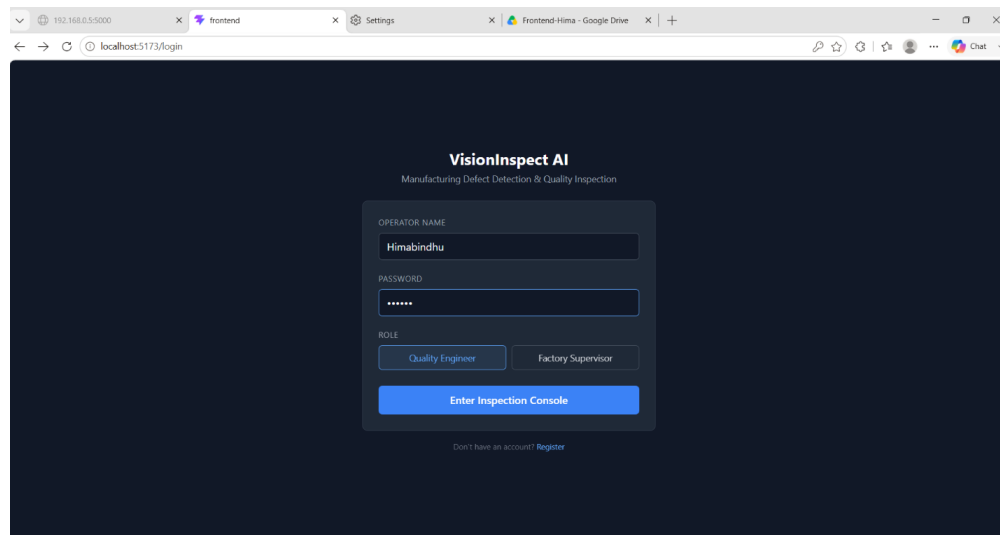
6. Frontend — Himabindhu Ravuri.

6.1 Tech Stack

- React (Vite), Tailwind CSS, React Router, React Context API, JWT auth against the real backend

6.2 Login Page

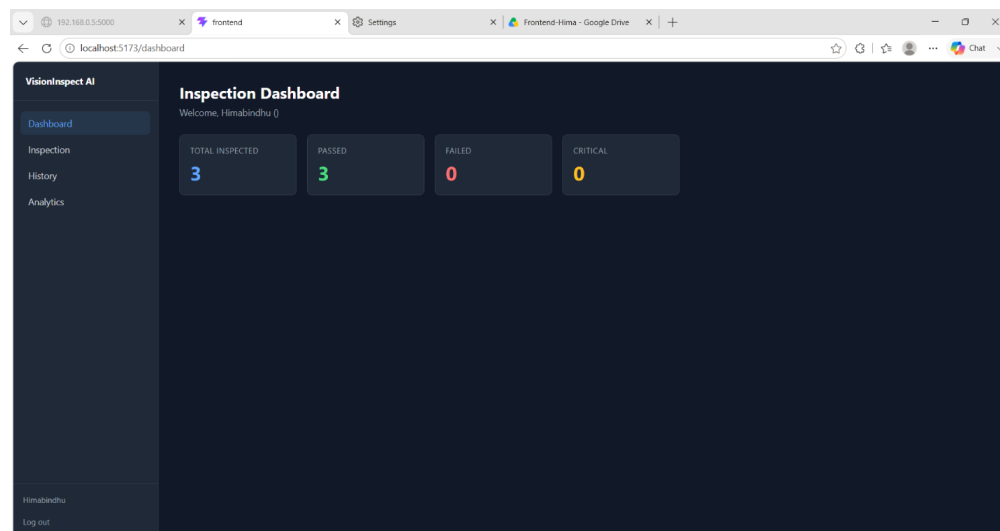
Role selection (Quality Engineer / Factory Supervisor); started on a placeholder login, then connected to Lakshyatha's real JWT login endpoint.



Login page — connected to the real backend login API (JWT)

6.3 Dashboard Page

Quick stats: total inspected, passed, failed, critical.



Inspection Dashboard — live summary stats

6.4 Inspection Page

Single & batch image upload with preview. Since the real model isn't connected yet, this currently returns mock results — but computed using the team's real severity formula (Size 30% + Location 25% + Defect Type 25% + Confidence 20%), so the numbers already behave correctly.

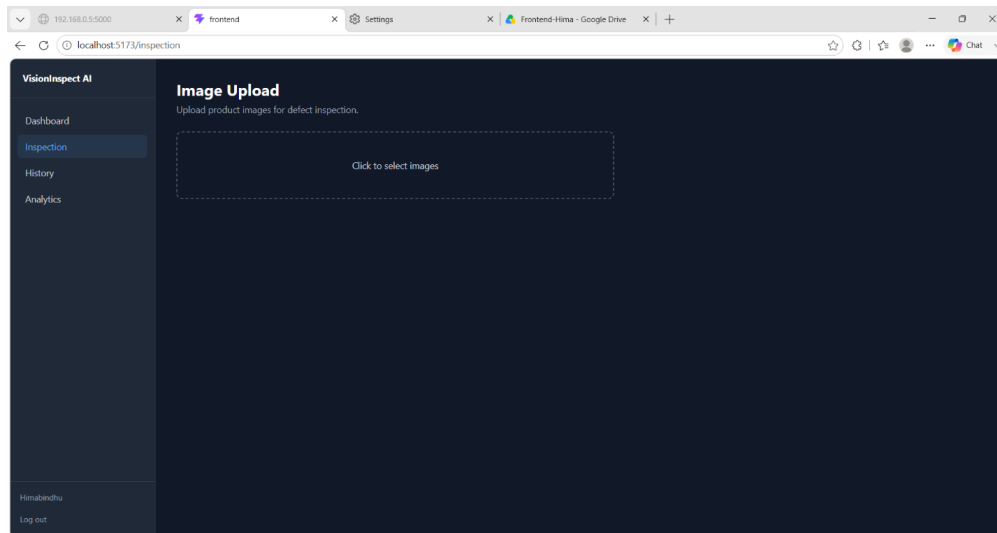


Image Upload / Inspection page

6.5 History Page

Full log of past inspections — file name, defect type, severity score, pass/fail result.

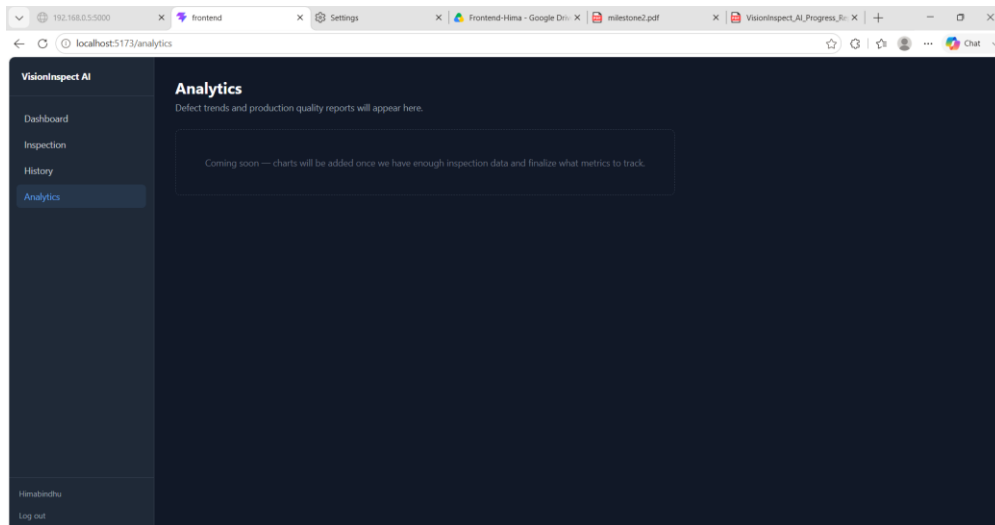
A screenshot of the 'Inspection History' page in the 'VisionInspect AI' application. The sidebar is the same as in the previous image, but the 'History' menu item is highlighted. The main content area is titled 'Inspection History' with the subtitle 'Full record of past inspections.' It contains a table with the following data:

FILE	DEFECT	SEVERITY	RESULT
002.png	Surface Scratch	Medium - 46	PASS
001.png	Dent	Medium - 58	PASS
000.png	Surface Scratch	Medium - 58	PASS

Inspection History — full record table

6.6 Analytics Page

Placeholder for now — will show real defect trends and quality reports once live inspection data is flowing.



Analytics page — placeholder, pending real data

6.7 Architecture Choice

All backend calls are isolated into a single file (`api.js`). Login → real JWT token stored; upload → sent for checking (currently mock); result → shown & saved to history; dashboard/history just read that saved data. This means when the real model and backend are fully live, only `api.js` needs to change — no page rewrites.

6.8 Known Limitations (Honest, Not Hidden)

- Inspection results are still mock data — AI model not connected yet
- Analytics will get real charts once there's real inspection data

7. What Matches Milestone 1 Requirements

Milestone 1 asked for: project initialization & architecture setup; authentication & image upload workflows; a working inspection dashboard; dataset integration & preprocessing.

Requirement	Status	
Architecture & folder structure	Done	Arnab / Yash
Database schema (5 tables, PostgreSQL)	Done	Arnab
Backend REST API (Flask, 12 endpoints)	Done	Lakshyatha
JWT authentication (backend + frontend)	Done	Lakshyatha / Himabindhu
Image upload — single & batch	Done	Himabindhu / Lakshyatha
Inspection dashboard (UI)	Done	Himabindhu
Dataset load & full exploration (15 categories)	Done	Yash
Dataset preprocessing (all 15 categories)	Done	Yash (+ Arnab verified)
Role-based access control enforcement	Done	Frontend + team decision

8. Next Steps — Milestone 2

- Train PatchCore across all 15 categories (not just Bottle) and generate real defect predictions.
- Replace the frontend's mock inspection results with live calls to the trained model via the backend.
- Wire the backend's AI-integration services to the actual model checkpoint.
- Add mangodb to the data management.
- Deploy the database to a managed cloud PostgreSQL instance (e.g. Supabase/Render).
- Build the real analytics dashboard once live inspection data is flowing.